01 — Introduction to PHP

What PHP is

PHP is a language that runs **on the server**. The browser asks for a page, PHP runs, produces HTML, and only that HTML is sent back. The visitor never sees your PHP code.

```
Browser  --request-->  Web server + PHP  --runs page.php-->  HTML
Browser  <--HTML-----  Web server
```

A few facts worth knowing:

- Created in 1994 by Rasmus Lerdorf ("Personal Home Page", now "PHP: Hypertext Preprocessor").
- Powers a large share of the web: WordPress, Drupal, Magento, Laravel and Symfony apps.
- PHP 7 (2015) made it about twice as fast. PHP 8 (2020+) added modern features: named arguments, `match`, enums, readonly properties, union types.
- Dynamically typed, like Python, with optional type declarations, a bit like Java.

Configuring the engine

PHP reads its settings from a text file called **`php.ini`**, once, when it starts.

Finding it

```
php --ini
```

```
Configuration File (php.ini) Path: /etc/php/8.3/cli
Loaded Configuration File:         /etc/php/8.3/cli/php.ini      <- the file to edit
Scan for additional .ini files in: /etc/php/8.3/cli/conf.d       <- extra files, loaded after
```

If *Loaded Configuration File* says `(none)`, PHP runs on its built-in defaults: create a `php.ini` in the indicated path by copying `php.ini-development` (shipped with PHP).

Typical locations:

Install	php.ini
Windows (zip)	`C:\php\php.ini`
macOS (Homebrew)	`/opt/homebrew/etc/php/8.x/php.ini` (Intel Macs: `/usr/local/etc/php/8.x/`)
Linux (Debian/Ubuntu)	`/etc/php/8.x/cli/php.ini`
Laravel Herd	open it from Herd's settings (PHP tab)

> On a real server, the **command line and the web server can use different files**. On Linux: `/etc/php/8.x/cli/php.ini` for `php` in the terminal, `/etc/php/8.x/fpm/php.ini` for the website. Always check with `phpinfo()` in a page, not only `php --ini`. With `php -S` (our setup), it's the same file.

PHP ships two templates: `php.ini-development` (shows errors, verbose) and `php.ini-production` (hides errors, safer). Start from the one matching the machine.

Syntax

```ini
; A comment starts with a semicolon
display_errors = On
error_reporting = E_ALL
memory_limit = 256M
date.timezone = Europe/Paris
upload_max_filesize = 10M
post_max_size = 12M          ; must be >= upload_max_filesize

extension=pdo_sqlite         ; enable an extension (remove the ";" to activate a commented one)
```

After editing, restart PHP: stop `php -S` with `Ctrl+C` and start it again (on a server: restart Apache or PHP-FPM). Then

check the value in `phpinfo()`: it shows the *Local* and *Master* value of each setting.

Settings you'll meet most often

Setting	Meaning	Dev value	Production
`display_errors`	Show errors in the page	`On`	`Off`
`log_errors` / `error_log`	Write errors to a log file	`On` / path	`On` / path
`error_reporting`	Which errors are reported	`E_ALL`	`E_ALL`
`memory_limit`	Max memory per script	`128M` (default)	as needed
`max_execution_time`	Max seconds per request	`30` (default)	as needed
`upload_max_filesize`	Max size of one uploaded file	`2M` (default)	as needed
`post_max_size`	Max size of a whole POST request	`8M` (default)	≥ upload size
`date.timezone`	Default timezone for `date()`	`Europe/Paris`	same
`session.cookie_httponly`	Session cookie hidden from JavaScript	`On`	`On`

Three ways to change a setting

From the widest to the narrowest scope:

Where	Applies to	Example
`php.ini`	everything using this PHP	`memory_limit = 256M`
Command-line flag `-d`	this one run / this server	`php -S localhost:8000 -d display_errors=1`
`ini_set()` in code	this script only, from that line on	`ini_set('display_errors', '1');`

Read the current value from code with `ini_get('memory_limit')`.

Trap: not every setting can be changed with `ini_set()`. Settings needed *before* your script runs, like `upload_max_filesize` and `post_max_size` (the upload has already happened when your code starts), can only be set in `php.ini`, with `-d`, or in server config. `ini_set()` then silently returns `false`:

```php
var_dump(ini_set('upload_max_filesize', '10M'));   // bool(false): ignored!
var_dump(ini_get('upload_max_filesize'));          // string(2) "2M"
```

The PHP manual lists each setting's scope (`PHP_INI_ALL` = changeable anywhere, `PHP_INI_PERDIR` = not from `ini_set`, `PHP_INI_SYSTEM` = `php.ini` only).

Inspecting the configuration

```
php -m                    # loaded extensions
php -i                    # all settings (long! filter it:)
php -i | grep memory_limit             # macOS / Linux
php -i | findstr memory_limit          # Windows
```

In a web page, `<?php phpinfo();` shows everything as a page. **Never leave it on a public site**: it reveals your configuration.

Extensions

Much of PHP's functionality lives in **extensions**: modules that can be switched on or off. A few you'll meet constantly:

Extension	Provides	Typical use
`pdo_sqlite`, `pdo_mysql`, `pdo_pgsql`	database drivers	modules 05 and 06
`mbstring`	`mb_*` functions	strings with accents, emojis
`intl`	`NumberFormatter`, `IntlDateFormatter`, `Collator`	prices and dates in the user's language; required by Symfony, WooCommerce…
`gd` (or `imagick`)	image creation and resizing	thumbnails, captchas
`zip`	`ZipArchive`	create or read .zip files; required by Composer
`curl`	HTTP requests	calling APIs
`fileinfo`	real MIME type of a file	upload checks (module 04)
`opcache`	caches compiled code	performance, always on in production
`xdebug`	step debugger, profiler	development only

Is it enabled?

```
php -m                          # list of loaded extensions
php -m | grep intl              # macOS / Linux
php -m | findstr intl           # Windows
php --ri intl                   # details of one extension
```

From code: `extension_loaded('intl')` returns `true` or `false`. In a page, `phpinfo()` has one section per extension.

A missing extension shows up as an error like "*Call to undefined function imagecreatetruecolor()*" or "*Class "NumberFormatter" not found*".

Enabling or installing one

Extensions fall into two groups: **bundled** ones, shipped with PHP and just waiting to be switched on, and **third-party** ones (xdebug, redis, imagick…) that must be downloaded first.

Windows (zip install) — the bundled extensions are already in `C:\php\ext\` as `php_xxx.dll`. Enable them in `php.ini`:

```
extension_dir = "ext"
extension=intl
extension=gd
extension=zip
```

For a third-party extension, download the DLL matching **exactly** your PHP version, *x64* and *NTS* (Non Thread Safe), put it in `ext\`, and add the line. Xdebug's site has a wizard that tells you which file to download: paste the output of `php -i` into it.

macOS (Homebrew) — `intl`, `gd`, `zip`, `curl`, `mbstring` and the PDO drivers are compiled in: nothing to do. Third-party ones install with `pecl install xdebug` (or `redis`, `imagick`…), which also adds the line to `php.ini`.

Linux (Debian/Ubuntu) — one package per extension, enabled automatically:

```
sudo apt install php-intl php-gd php-zip php-curl
# disable / re-enable without uninstalling:
sudo phpdismod xdebug
sudo phpenmod xdebug
```

Laravel Herd — ships with the common extensions already enabled.

After any change: **restart PHP** (`php -S`, PHP-FPM or Apache), then check with `php -m`.

> `extension=` vs `zend_extension=`: a few extensions hook deeper into the engine and are loaded with `zend_extension=` instead. The two you'll see are `opcache` and `xdebug`.

> **PECL → PIE**: `pecl` is the historical installer for third-party extensions. The PHP Foundation now provides **PIE** (PHP Installer for Extensions), which works like Composer: `pie install xdebug/xdebug`. You'll find both in tutorials.

PHP inside HTML

A `.php` file is HTML by default. PHP code goes between `<?php` and `?>`.

```
<!DOCTYPE html>
<html>
<body>
    <h1><?php echo "Hello"; ?></h1>
    <p>Today is <?= date('l') ?></p>   <!-- <?= ... ?> is a shortcut for echo -->
</body>
</html>
```

A file containing **only** PHP (classes, scripts) starts with `<?php` and should **not** end with `?>`: this avoids sending stray whitespace by accident.

Instructions end with `;`. Comments: `// one line`, `# one line`, `/* several lines */`.

Variables

Variables start with `$`. No declaration, no type needed.

```
$name = "Alice";
$age = 32;
$price = 19.99;
$isAdmin = false;
$nothing = null;
```

The four scalar types: `string`, `int`, `float`, `bool`. Plus `array`, `object` and `null`.

```
var_dump($age);       // int(32)  -> type AND value, your best debugging friend
echo gettype($price); // double (historical name for float)
```

Strings

```
$first = 'Alice';
echo 'Hello $first';      // Hello $first   (single quotes: no interpolation)
echo "Hello $first";      // Hello Alice    (double quotes: variables replaced)
echo "Hello {$first}!";   // braces: safer when text follows
echo 'Hello ' . $first;   // . is the concatenation operator (not +)
```

Constants

```
const VAT_RATE = 0.2;           // preferred, at top level or in classes
define('SITE_NAME', 'My shop'); // older style, works anywhere at runtime

echo VAT_RATE;                  // no $
```

Built-in constants: `PHP_VERSION`, `PHP_EOL` (line break), `PHP_INT_MAX`.

Type juggling and casting

PHP converts types automatically when needed. That's handy and sometimes surprising.

```
var_dump("5" + 3);      // int(8)    the string is converted to a number
var_dump("5" . 3);      // string(2) "53"
var_dump(10 / 4);       // float(2.5)
var_dump(intdiv(10, 4)); // int(2)
```

Explicit conversion (casting):

```
$n = (int) "42abc";   // 42
$f = (float) "3.14";  // 3.14
$s = (string) 12;     // "12"
$b = (bool) "";       // false
```

Values that are `false` when converted to bool: `false`, `0`, `0.0`, `""`, `"0"`, `[]`, `null`. Everything else is `true`.

`==` vs `===`

```php
var_dump(0 == "0");     // true  : values compared after conversion
var_dump(0 === "0");    // false : value AND type must match
var_dump(null == false); // true
```

Rule: use `===` and `!==` by default. Use `==` only when you mean it.

Strict types

Add this at the very top of a file to forbid automatic conversion when calling typed functions:

```php
<?php
declare(strict_types=1);
```

We'll see typed functions in module 02.

Now do the exercises.

01 — Exercises

Create your files in a working folder of your own (for example `my-work/01/`).

1.1 — Explore and change your configuration

1. Run `php --ini` and find the path of the loaded `php.ini`. If it says `(none)`, create it from `php.ini-development` (ask the trainer if you can't find it).
2. Run `php -m` and check that `pdo_sqlite` is in the list.
3. Create `info.php` containing `<?php phpinfo();`, start `php -S localhost:8000` in that folder and open http://localhost:8000/info.php. Note the values of `display_errors`, `upload_max_filesize` and `date.timezone`.
4. Open your `php.ini` in your editor and set:

   ```
   date.timezone = Europe/Paris
   upload_max_filesize = 10M
   post_max_size = 12M
   ```

   Restart `php -S` (`Ctrl+C`, then start it again) and reload `info.php`: the new values must appear.
5. Create `ini.php`:

   ```php
   <?php
   var_dump(ini_get('memory_limit'));
   var_dump(ini_set('memory_limit', '256M'));
   var_dump(ini_get('memory_limit'));

   var_dump(ini_set('upload_max_filesize', '50M'));
   var_dump(ini_get('upload_max_filesize'));
   ```

   Run it with `php ini.php`. Why does the first `ini_set` work and not the second?
6. Run `php -d memory_limit=64M ini.php`. What changed in the first line, and why?

No solution file: the answers are in the lesson. Compare with the trainer.

1.2 — Business card page

Create `card.php`, a web page that displays a business card.

- Store your first name, last name, job and year of birth in variables.
- Store the current year with `date('Y')`.
- Display: the full name in an `<h1>`, the job, and the age (computed).
- Use both `<?php echo ... ?>` and the short form `<?= ... ?>` at least once.

Run it with `php -S localhost:8000` and open http://localhost:8000/card.php.

1.3 — Type detective

Create `types.php`, a command-line script (`php types.php`).

1. Predict, **on paper first**, the result of each line, then run with `var_dump()` to check:

   ```php
   "10" + 5
   "10" . 5
   "3 apples" + 2      // PHP 8 gives a warning here: read it
   7 / 2
   intdiv(7, 2)
   7 % 2
   (int) "12.9"
   (bool) "0"
   (bool) "false"
   0 == ""
   0 === ""
   null ?? "default"
   ```

2. Define a constant `VAT_RATE` (0.2) and a variable `$priceExclTax` (49.90). Display the price including tax, rounded to 2 decimals with `round()`.

3. Display your result with `number_format($value, 2, ',', ' ')` to get the French format (`59,88`).

1.4 — Extensions

1. Check whether `intl`, `gd` and `zip` are enabled on your machine (`php -m`). Enable or install the missing ones, following the lesson for your OS.
2. Create `extensions.php` (command line) that, for each of the three, displays `intl: OK` or `intl: MISSING` using `extension_loaded()`.
3. Then use each one:
   - **intl**: display `1234567.891` as a price in euros, formatted for France (`1 234 567,89 €`) and for the US (`€1,234,567.89`). Use `new NumberFormatter('fr_FR', NumberFormatter::CURRENCY)` and its `formatCurrency()` method.
   - **gd**: create a 200×100 PNG image with a colored background and a line of text, saved as `badge.png`. Functions: `imagecreatetruecolor`, `imagecolorallocate`, `imagefill`, `imagestring`, `imagepng`.
   - **zip**: create `archive.zip` containing `badge.png` and a text file built from a string. Class: `ZipArchive` (`open` with `ZipArchive::CREATE`, `addFile`, `addFromString`, `close`).